

# Exercise series 2: Solutions

Introduction to databases

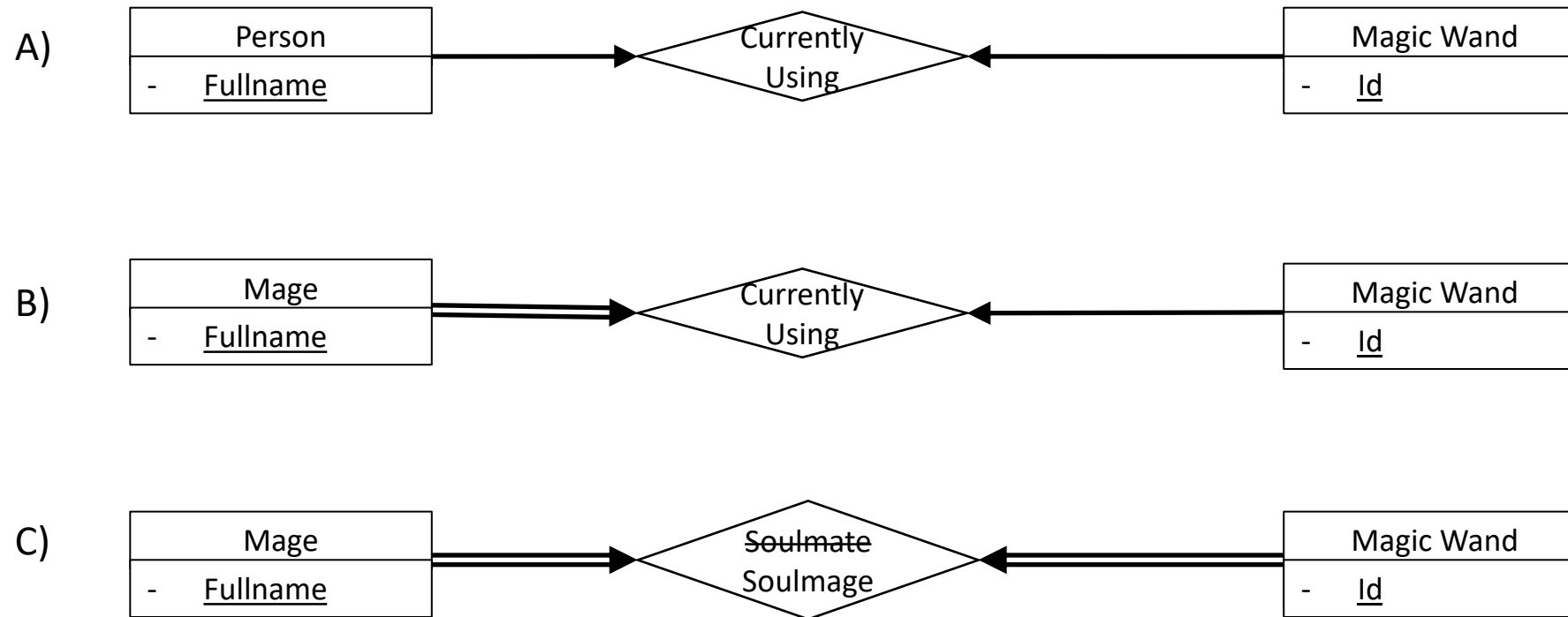


**ADReM**  
Advanced Database Research & Modelling  
University of Antwerp

 Universiteit  
Antwerpen

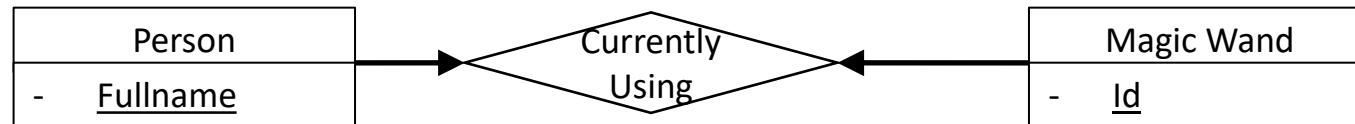
# Problem 1

Using NOT NULL, UNIQUE, FOREIGN KEY and PRIMARY KEY translate these relationships into tables



# Solution A-1

<https://www.db-fiddle.com/f/pJFSAqy67pkiW5JLX9CgFt/0>

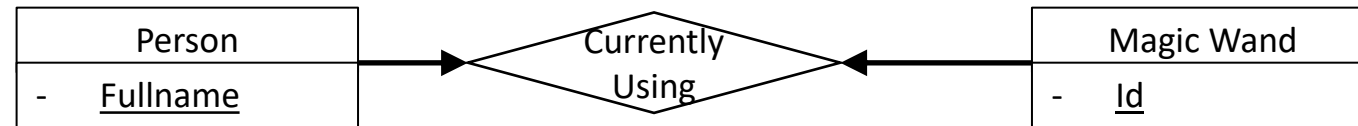


```
CREATE TABLE Person (  
    fullname VARCHAR(255) PRIMARY KEY  
);  
  
CREATE TABLE Magic_Wand (  
    id INT PRIMARY KEY  
);  
  
CREATE TABLE Currently_using (  
    wid INT PRIMARY KEY REFERENCES Magic_Wand(id),  
    fullname VARCHAR(255) NOT NULL UNIQUE REFERENCES Person(fullname)  
);
```



# Solution A-2

<https://www.db-fiddle.com/f/pJFSAqy67pkiW5JLX9CgFt/1>

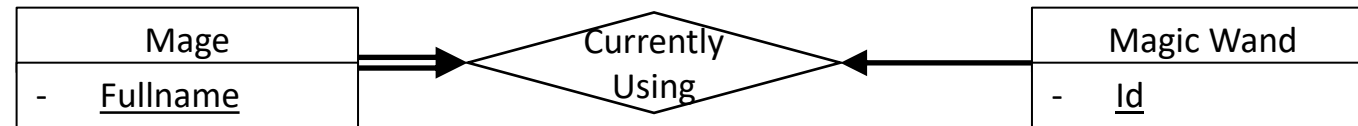


```
CREATE TABLE Person (  
    fullname VARCHAR(255) PRIMARY KEY  
);  
  
CREATE TABLE Magic_Wand (  
    id INT PRIMARY KEY,  
    mage VARCHAR(255) UNIQUE REFERENCES Person(fullname)  
);
```



# Solution B

<https://www.db-fiddle.com/f/pJFSAqy67pkiW5JLX9CgFt/2>



```
CREATE TABLE Magic_Wand (  
  id INT PRIMARY KEY  
);
```

```
CREATE TABLE Mage (  
  fullname VARCHAR(255) PRIMARY KEY,  
  wid INT UNIQUE NOT NULL REFERENCES Magic_Wand(id)  
);
```

Pay attention where to put  
the foreign key!

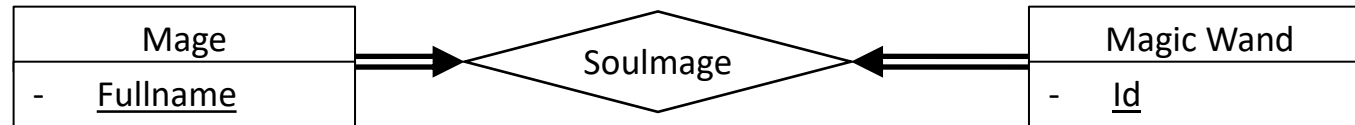


ADReM

Advanced Database Research & Modelling  
University of Antwerp

Universiteit  
Antwerpen

# Solution C



```
CREATE TABLE Soulmage(  
  mage VARCHAR(255) PRIMARY KEY,  
  wid INT UNIQUE NOT NULL  
);
```

Formally correct, but  
BLEARGH...



# Problem 2

1. Convert the ER model to the relation model: CREATE TABLE for all entities, CREATE TABLE for many-to-many-relations, and FOREIGN KEY attributes for 1-to-many relations
2. Add example data (entity and many-to-many) to each table using INSERT or UPDATE
3. Retrieve all tuples in table with SELECT: SELECT \* FROM tablename;



# Problem 2

**Users** are uniquely identified by an email address. For each user we also maintain the first and last name and date of birth. Users can create **messages**, where we keep track of the user that created the message and the date and time. A message has a body (text, possibly using html encoding) and zero or more attached **files**.

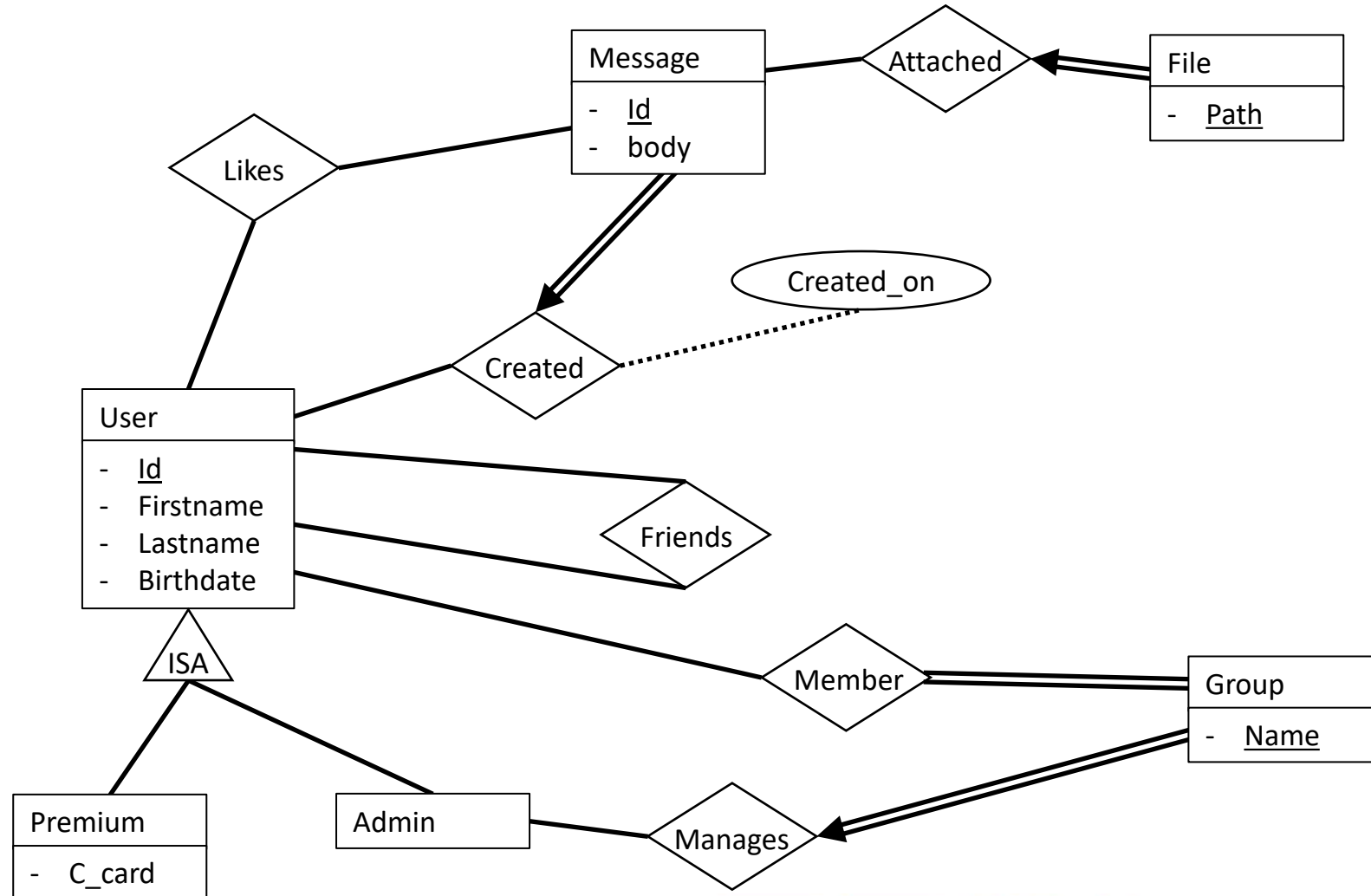
Users can also be a member of a **group**. A group has name and is managed by special user who is the group **administrator**. Users can become **friends** with other users. User A asks user B to become friends. After user B accepts the invitation, the friendship status and start date is stored. We also store which friends **like** each message.

Users can also upgrade to a **premium** account with more privileges. For premium users we also store the credit card number.





# ER model



# ER to Relational Model

```
CREATE TABLE User_table(  
    id INT,  
    firstname VARCHAR(255),  
    lastname VARCHAR(255),  
    birthdate DATE,  
    PRIMARY KEY (id)  
);
```

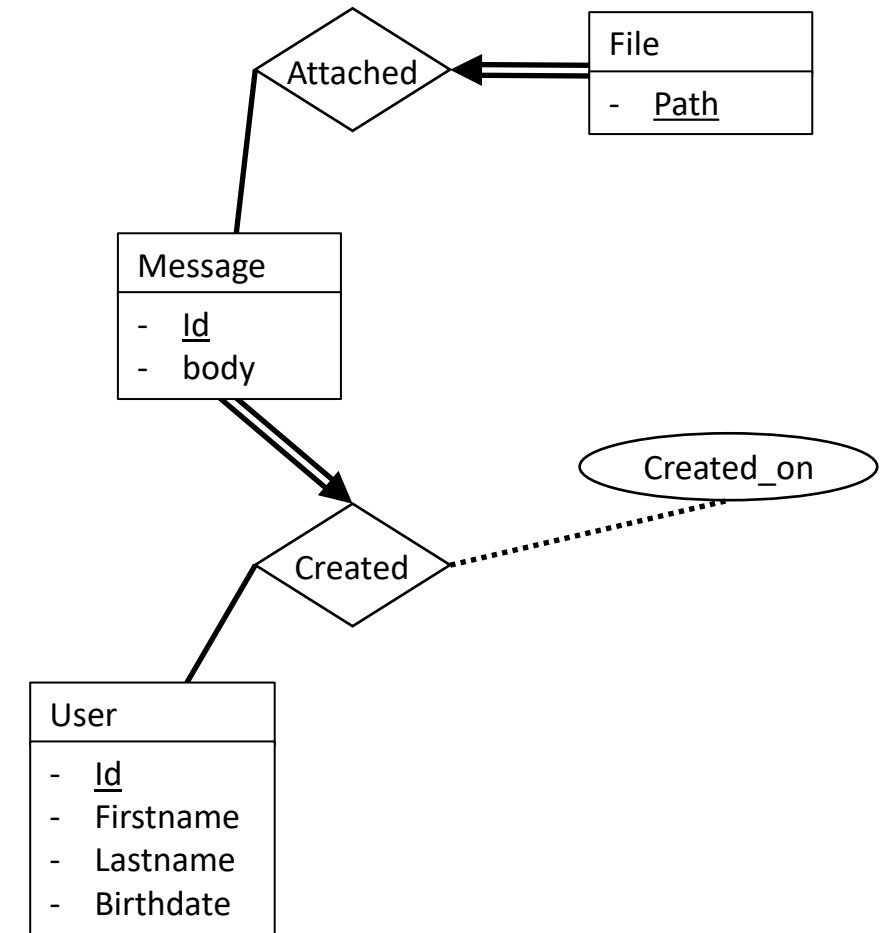
| User        |
|-------------|
| - <u>Id</u> |
| - Firstname |
| - Lastname  |
| - Birthdate |



# ER to Relational Model

```
CREATE TABLE Message(  
  id INT PRIMARY KEY,  
  body VARCHAR(255),  
  uid INT REFERENCES User_table(id) NOT NULL,  
  created_on TIME  
);
```

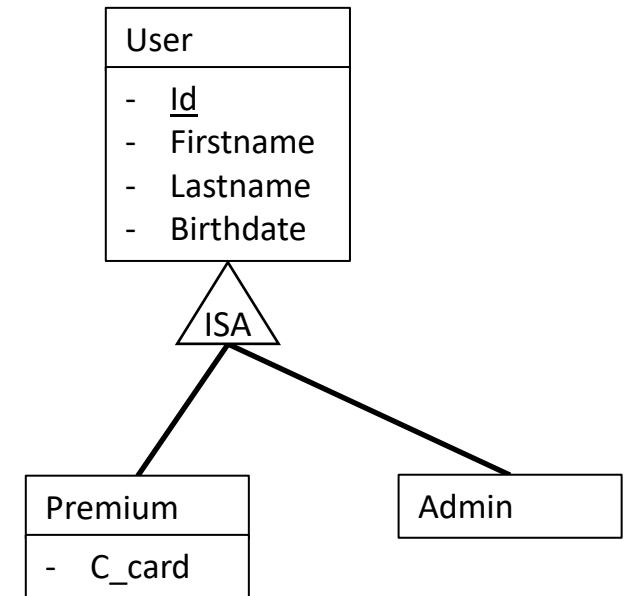
```
CREATE TABLE File_table(  
  file_path VARCHAR(255) PRIMARY KEY,  
  mid INT NOT NULL REFERENCES Message(id)  
);
```



# ER to Relational Model

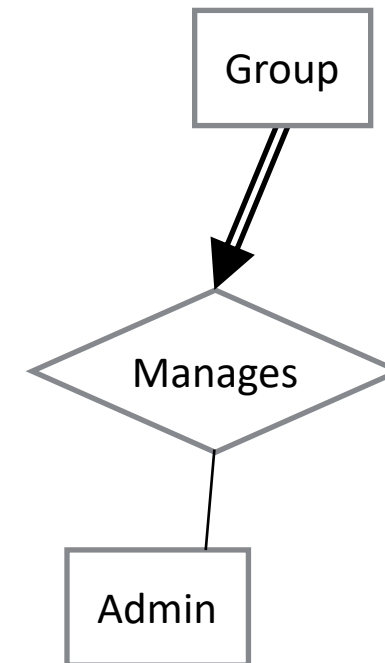
```
CREATE TABLE Admin_table(  
    id INT PRIMARY KEY REFERENCES User_table (id)  
);
```

```
CREATE TABLE Premium(  
    id INT,  
    c_card VARCHAR(255),  
    PRIMARY KEY (id),  
    FOREIGN KEY (id) REFERENCES User_table(id)  
);
```



# ER to Relational Model

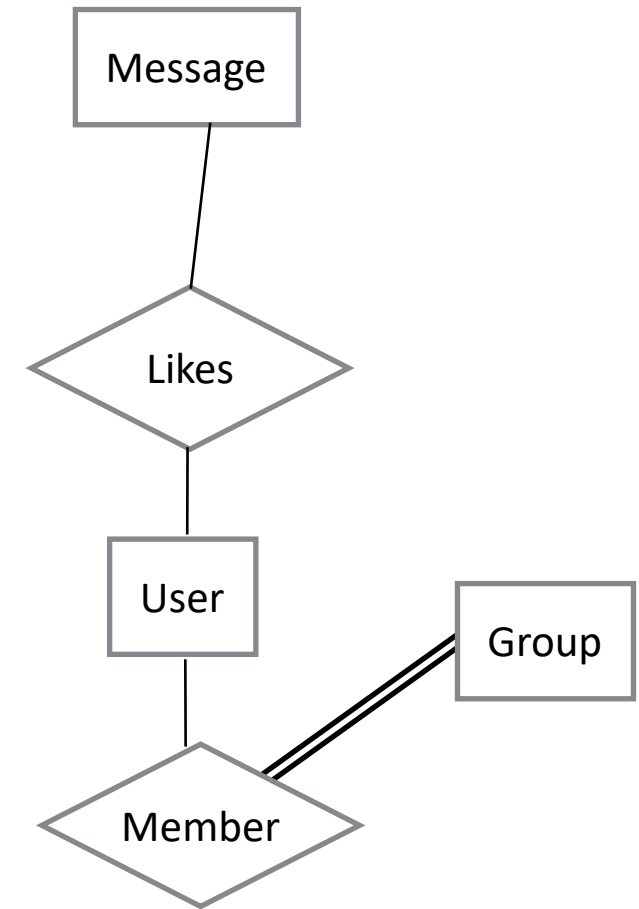
```
CREATE TABLE Group_table(  
  name VARCHAR(255) PRIMARY KEY,  
  aid INT NOT NULL REFERENCES Admin_table(id)  
);
```



# ER to Relational Model

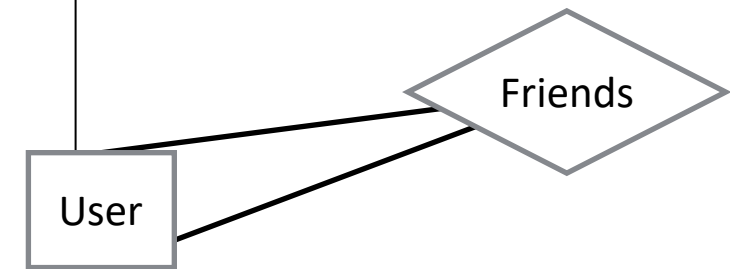
```
CREATE TABLE Member(  
  user_id INT REFERENCES User_table(id),  
  group_name VARCHAR(255) REFERENCES  
  Group_table(name),  
  PRIMARY KEY(user_id, group_name)  
);
```

```
CREATE TABLE Likes(  
  user_id INT REFERENCES User_table(id),  
  message_id INT REFERENCES Message(id),  
  PRIMARY KEY(user_id, message_id)  
);
```



# ER to Relational Model

```
CREATE TABLE Friends(  
  user_id1 INT,  
  user_id2 INT,  
  PRIMARY KEY (user_id1, user_id2),  
  FOREIGN KEY (user_id1) REFERENCES User_table(id),  
  FOREIGN KEY (user_id2) REFERENCES User_table(id)  
);
```



# Populating the database

## Example tuples:

user(1, 'Charlie', 'Carter', '1984-05-17'), user(2, 'Alice', 'Miller', '1989-01-01')

user(3, 'Bob', 'Moore', '1991-11-30') , premium(3, '000-0000-000')

friends(1, 2), friends(1, 3), friends(2, 3)

message(1001, 'Charly #loving-posgreSQL!', 1, '2017-11-10 12:00:00'),

likes(3, 1001), likes(2, 1001)

message(1002, 'Alice thinks #databases rocks', 2, '2017-11-10 12:00:00')

likes(1, 1002)

admin(1)

group('UA-databases-lovers', 1)

member(1, 'UA-databases-lovers')

member(2, 'UA-databases-lovers')

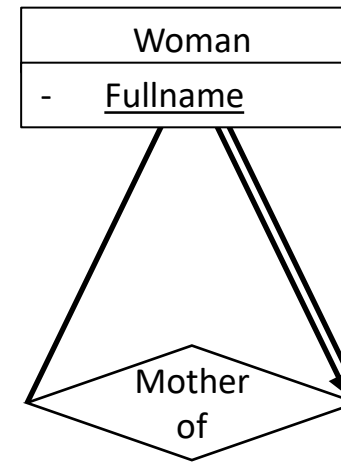
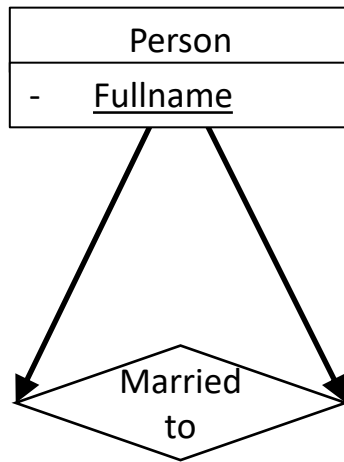
member(3, 'UA-databases-lovers')



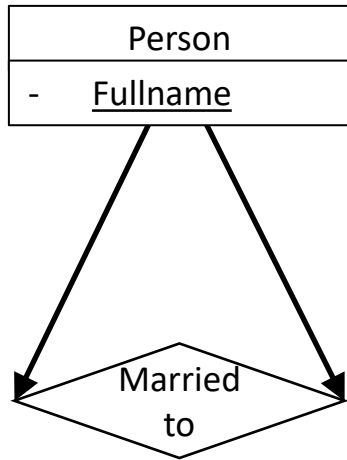


# Problem 3

All the following ER-diagrams have problems when we try to translate them: find those problems



# Solution 3.1



```
CREATE TABLE Person(  
    fullname TEXT PRIMARY KEY  
);
```

```
CREATE TABLE Married_to(  
    fullname_1 TEXT UNIQUE REFERENCES Person,  
    fullname_2 TEXT UNIQUE REFERENCES Person,  
    PRIMARY KEY (fullname_1, fullname_2)  
);
```

```
INSERT INTO Person VALUES  
('Hermia'), ('Lysander'), ('Demetrius'), ('Helena');
```

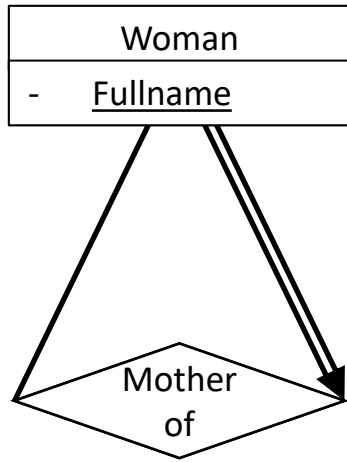
```
INSERT INTO Married_to VALUES  
('Lysander', 'Helena'),  
('Helena', 'Demetrius'),  
('Demetrius', 'Hermia'),  
('Hermia', 'Lysander');
```

E.G.: *A Midsummer Night's Dream*

**PROBLEM: Married\_to is NOT symmetric!**



# Solution 3.2



```
CREATE TABLE Woman(  
    fullname TEXT PRIMARY KEY,  
    mother TEXT NOT NULL REFERENCES Woman  
);
```

```
INSERT INTO Woman VALUES  
    ('Elisabeth', 'Charlotte'),  
    ('Franziska', 'Charlotte'),  
    ('Charlotte', 'Elisabeth');
```

E.G.: *Dark*

**PROBLEM: there is a cycle somewhere!**



**ADReM**  
Advanced Database Research & Modelling  
University of Antwerp

**Universiteit  
Antwerpen**